

SIMATS Engineering — Assessment Tool 2

Course: Artificial Intelligence (CSA17) | CO2: Search & Game-Solving Techniques

Answer Key — Annexure A: Scenario Based Assignment

Q1. Flood-Relief Drone Routing — Greedy Best-First Search vs. A*

i. Greedy Best-First Search Behaviour

Greedy Best-First Search expands the node with the smallest heuristic value $h(n)$ (straight-line distance to the destination) and completely ignores the cost already spent, $g(n)$. In this flood scenario the drone would always fly toward the option that looks geometrically closest to the delivery point, without accounting for how much extra fuel/time bad weather or difficult terrain adds along that direction.

Strengths:

- Very fast — expands far fewer nodes, which suits the drone's urgency and limited onboard compute.
- Works well when the heuristic is a reasonably good proxy for real difficulty and the environment is mostly static.

Weaknesses under uncertainty:

- Not optimal — it can be lured toward a route that looks closest in straight-line distance but is actually blocked or slowed by weather, wasting time on a costlier real path.
- It never reconsiders cost incurred so far, so once committed to a promising-looking direction it can fly straight into a dead end (blocked road) with no built-in cost-based safeguard.
- Because it discounts $g(n)$ entirely, it is especially fragile in a dynamic environment where the real cost of a segment can spike suddenly (flooded road, storm cell) after the heuristic has already pulled the drone that way.

ii. How A Handles the Same Situation*

A* expands nodes by $f(n) = g(n) + h(n)$: the actual accumulated cost of the route flown so far, plus the heuristic estimate of the remaining straight-line distance. This lets it weigh the two factors against each other — a route with a smaller straight-line remainder is not preferred if reaching it has already or would incur a much higher real weather/terrain cost. As long as the straight-line-distance heuristic never overestimates true remaining cost (which holds for physical distance vs. terrain-inflated travel cost), A* is guaranteed to find the minimum-cost route through the known graph at the time of planning.

iii. Impact of Heuristic Accuracy

For Greedy Best-First Search the heuristic is the *only* thing driving the search, so an inaccurate heuristic (straight-line distance that ignores flooded/blocked roads) can send the drone down badly suboptimal or even unsafe paths with no correction mechanism. For A*, heuristic accuracy affects efficiency more than correctness: a more informed (but still admissible) heuristic prunes the search faster and expands fewer nodes, while a weak/underestimating heuristic (like plain straight-line distance in a terrain with large weather-driven cost swings) makes A* expand more nodes — degrading toward Uniform-Cost Search behaviour — but it still returns the optimal route, just more slowly.

iv. Effect of Dynamic Changes (Blocked Paths)

Both algorithms plan over a snapshot of the graph; if a path becomes blocked after planning, the previously computed route is invalidated and must be replanned. Frequent changes force repeated searches, which is cheap for Greedy Best-First Search (few nodes expanded) but can be expensive for A* if the graph is large. This is precisely why a purely offline application of either algorithm is insufficient here

— the system needs an online/incremental replanning loop (e.g. repeated A*, or a dynamic variant such as D* Lite) that reacts to real-time blocked-path updates instead of a single one-shot plan.

v. Recommendation

A* (or an incremental/replanning variant of it, such as D* Lite) is recommended. Given that the cargo is emergency medicine, safety and near-optimal cost matter more than shaving off a little search time, which rules out pure Greedy Best-First Search. A* balances the urgency requirement (heuristic-guided, faster than uninformed search) with reliability (uses real accumulated cost, so it correctly accounts for weather and terrain penalties), and its incremental/anytime variants support the fast replanning needed when paths become blocked mid-flight — the right trade-off between real-time constraints, optimality, and safety.

Q2. Smart-City Traffic Signal Optimisation

Formulation as an Optimisation Problem

State: a complete configuration of signal-timing parameters (phase durations/offsets) across all intersections. **Objective function:** a weighted combination to minimise — $\text{Cost} = w_1 \cdot (\text{total waiting time}) + w_2 \cdot (\text{fuel consumption}) + w_3 \cdot (\text{congestion level})$. **Neighbourhood:** configurations reachable by adjusting one or a few signals' timings. There is no single fixed goal state to search toward, so this is a continuous/combinatorial optimisation problem rather than a path-finding problem.

Why traditional (tree) search is inefficient: classic search algorithms (BFS/DFS/A*) are designed to find a path from a start to a well-defined goal state in a graph small enough to explore systematically. Here the state space is astronomically large (many intersections $\times$ many possible timing values each), there is no discrete goal test (only a value to minimise), and traffic patterns change continuously — making an exhaustive or systematic tree search computationally infeasible and conceptually mismatched. Local search and metaheuristics, which search directly over complete configurations and iteratively improve them, fit this kind of problem far better.

i. Hill Climbing and Its Limitations

Hill Climbing starts from an initial timing configuration and repeatedly moves to a neighbouring configuration (e.g. tweak one intersection's green-light duration) that improves the objective, stopping when no neighbour is better. It is simple and cheap per step, but has no mechanism to escape a configuration that looks locally best yet is far from the global optimum, and no way to look ahead when a single-variable change doesn't help even though a coordinated multi-signal change would.

ii. Local Maxima, Plateaus, and Ridges

- **Local maxima:** a set of signal timings that looks best among its immediate neighbours (e.g. one intersection's timing seems ideal) but a better city-wide "green wave" coordination exists that requires changing several signals together — Hill Climbing stops early and misses it.
- **Plateaus:** flat regions where small timing tweaks make no measurable difference to waiting time (e.g. traffic is already near saturation, so minor changes don't register) — Hill Climbing sees no improving direction and halts prematurely even though better regions lie further away.
- **Ridges:** improvement requires simultaneously adjusting several correlated intersections along a corridor (e.g. synchronising a whole arterial road), but Hill Climbing only changes one variable at a time and cannot move diagonally along such a ridge, so it stalls or zig-zags inefficiently.

iii. Simulated Annealing / Genetic Algorithms as a Remedy

Simulated Annealing occasionally accepts a worse move with a probability controlled by a cooling temperature schedule, letting the search escape local maxima and plateaus early on, then settling into exploitation of good regions as the temperature drops. **Genetic Algorithms** maintain a population of candidate timing plans and use crossover and mutation to explore many regions of the search space in

parallel; crossover can combine good partial solutions from different parts of the population (e.g. good timings for one district combined with good timings for another), helping the search move along ridges and escape local optima that trap single-point methods like Hill Climbing.

iv. Recommended Approach

Genetic Algorithms are recommended as the primary optimiser for city-wide re-tuning, since their population-based, parallel exploration scales better to the huge combinatorial space of hundreds of intersections and adapts well as traffic patterns change (periodic re-runs of the GA against fresh traffic data). Simulated Annealing remains a good complementary tool for fast, localised fine-tuning of a small cluster of signals between full GA runs. This hybrid balances the scalability advantage of GA with the lighter-weight adaptability of SA for continuous, real-time adjustment.

Q3. Mars Rover — Online Search in Unknown Terrain

i. Why an Online Search Agent Is Needed

Offline search algorithms (BFS, A*, etc.) require the entire state graph to be known before planning a complete solution path. Here the terrain map is not known in advance — hazards like craters and rocks are discovered only when sensed. The rover therefore cannot compute a full path upfront; it must interleave acting, sensing, and (re)planning as it goes, which is precisely what an online search agent does.

ii. Challenges of Partial Observability and Dynamic Conditions

- Limited sensing range means hazards beyond the sensor horizon are invisible until the rover gets close, creating real risk of an unavoidable, unforeseen obstacle.
- A chosen direction may turn out to be a dead end only after committing several moves into it, forcing costly backtracking.
- Environmental changes (dust settling, small terrain shifts) can invalidate previously sensed information, so the rover cannot fully trust old data indefinitely and must periodically re-verify.

iii. Building and Updating the Internal Model

The rover maintains an internal map (e.g. an occupancy grid) with each cell marked free, hazard, or unknown. As it moves and senses its surroundings, newly observed cells are updated from unknown to free/hazard. When a previously assumed-clear cell turns out to be blocked, the map is corrected immediately and any current plan through that cell is discarded, triggering a re-plan from the rover's present position using the updated map.

iv. Online Search vs. Uninformed and Informed (Offline) Search

Aspect	Uninformed (BFS/UCS)	Informed/Offline (A*)	Online Search
Map knowledge required	Full graph known upfront	Full graph known upfront	Unknown; built as it explores
Guidance used	Path cost only	Path cost + heuristic	Local percepts + partial model, re-planned
Optimality guarantee	Yes, over known graph	Yes, if heuristic admissible	Not guaranteed — may need backtracking
Suited to this scenario	No — assumes full map	No — assumes full map	Yes — matches partial observability

v. Justification

An online search agent (e.g. an online variant of A*/UCS such as LRTA*, or a replanning method like D* Lite) is the most suitable choice: it directly addresses partial observability by sensing locally and updating knowledge incrementally, adapts to dynamic terrain by re-planning whenever new hazards are discovered, and prioritises safety by never committing to a long unseen plan — critical given the rover's high value, the communication delay to Earth that prevents real-time human correction, and the cost of an

irreversible mistake in hazardous, unknown terrain.

Q4. University Exam Timetabling as a CSP

i. Variables, Domains, and Constraints

Variables: one variable per course/exam that needs to be scheduled, e.g. Course1, Course2, ..., CourseN.

Domain of each variable: the set of feasible (time-slot, exam-hall) pairs it could be assigned, e.g. {(Day1-Slot1, HallA), (Day1-Slot1, HallB), (Day1-Slot2, HallA), ...}.

Constraints:

- **No overlapping exams for a student:** for any two courses that share at least one common student, their assigned time slots must differ — a binary constraint between such course pairs.
- **Limited exam halls:** at any given time slot, the number of exams scheduled cannot exceed the number of available halls, and each hall can host only one exam per slot (an all-different constraint over hall+slot combinations).
- **Precedence between subjects:** if subject A must precede subject B, then $\text{slot}(A) < \text{slot}(B)$ — an ordering constraint between specific course pairs.
- **Faculty availability:** a course requiring a specific invigilator/faculty can only be assigned a slot within that faculty's available time slots — restricts the effective domain of that course's variable.
- **Special accommodations:** students needing extra time or separate rooms impose extra constraints, e.g. certain exams must use accessible halls or be placed in slots that allow an extended duration without clashing with the student's other exams.

ii. How Backtracking Search Works Here

Courses are assigned one at a time to a (slot, hall) value from their domain. After each assignment, all constraints involving already-assigned courses are checked (no shared-student clash, hall capacity, precedence, faculty availability). If a violation is found, the search backtracks to the most recent assignment and tries the next available value in its domain. This continues until every course has a consistent assignment (solution) or all options are exhausted (failure). Ordering heuristics — such as assigning the most-constrained course first (most shared students) and trying the least-constraining (slot, hall) value first — substantially speed up the search.

iii. Constraint Propagation (Forward Checking)

After a course is assigned a (slot, hall), forward checking removes now-inconsistent values from the domains of related, not-yet-assigned courses — e.g. any course sharing a student with the just-assigned course loses that time slot from its domain, and any course from its remaining domain that used the now-occupied hall+slot pair loses that option too. This surfaces failures much earlier (a domain becoming empty signals an unavoidable dead end before deep, wasted backtracking), and shrinks the effective branching factor at every subsequent step — far more efficient than plain backtracking on a problem this size. Stronger propagation (e.g. arc-consistency via AC-3) can prune even further at extra computational cost per step.

iv. Real-World Challenges and Best Strategy for Scalability

As the number of courses and students grows, the number of shared-student constraints grows roughly with the square of enrolment overlaps, and resource constraints (limited halls, faculty windows) tighten domains further, making pure backtracking too slow for large universities. The scalable strategy combines: (1) backtracking with forward checking or full arc-consistency to prune early, (2) strong variable/value ordering heuristics (minimum-remaining-values and degree heuristic for variable choice, least-constraining-value for value choice), and (3) for very large instances, a switch to local-search repair

methods (e.g. min-conflicts, starting from a nearly-complete assignment and iteratively fixing violated constraints) which scale better than exhaustive backtracking once the problem size is very large.

Q5. Strategic Game AI — Minimax & Alpha-Beta Pruning

i. How Minimax Models the Problem

Minimax builds a game tree that alternates levels between the AI (the maximising player) and the human opponent (the minimising player), assuming the opponent always plays the move that is worst for the AI. Values are computed at terminal (or cutoff) nodes by a static evaluation function and propagated upward: MAX nodes take the largest value among their children, MIN nodes take the smallest, so the AI ultimately picks the move that guarantees the best outcome it can force even against optimal opposition.

ii. Computational Challenges of Large Game Trees

The number of nodes grows exponentially with search depth — roughly b^d for branching factor b and depth d . Real-time strategy games have a very large branching factor (many possible unit/action combinations per turn) and long games mean deep trees, so searching to terminal states within a strict time limit is computationally infeasible. This forces depth-limited search with a heuristic evaluation function at the cutoff, and real RTS games often add further complications (simultaneous moves, partial information) that pure Minimax does not natively model.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning tracks two bounds while traversing the tree: alpha (the best value the maximiser can already guarantee along the current path) and beta (the best value the minimiser can already guarantee). During search, if a node's value would make $\alpha \geq \beta$, the remaining siblings of that node cannot possibly change the final decision — a rational opponent would simply never allow that branch to be reached — so they are pruned without being evaluated. With good move ordering (trying likely-best moves first), Alpha-Beta can reduce the effective branching factor dramatically, in the best case searching to roughly twice the depth of plain Minimax for the same computational budget — directly helping meet the game's time limits.

iv. Designing an Evaluation Function

Since search must be cut off before reaching terminal states, a heuristic evaluation function estimates how favourable a state is by combining weighted factors such as:

- **Material/unit strength:** count and quality of units/resources each side controls.
- **Positional control:** map territory held, key choke points or high-ground occupied.
- **Economic strength:** resource income rate, number of production buildings.
- **Tempo/development:** tech/upgrade progress and production output relative to the opponent.
- **Threat and vulnerability:** exposure to imminent attacks or loss of key assets.
- **Mobility:** number of good actions/options currently available.

These are combined as a weighted sum, with weights tuned via playtesting or learning (e.g. self-play reinforcement learning) so the function approximates the true long-run value of a state closely enough to guide good decisions at the search cutoff.

v. Balancing Optimality and Real-Time Performance

The recommended strategy is **depth-limited Minimax with Alpha-Beta pruning and iterative deepening**: search progressively deeper, always keeping the best move found so far ready to play the instant the time budget expires (an anytime algorithm), combined with move-ordering heuristics (killer moves, history heuristic) to maximise how much Alpha-Beta can prune, and a well-tuned evaluation function at each cutoff. This trades guaranteed global optimality for near-optimal, time-bounded decisions — the right balance for an opponent AI that must respond within strict real-time limits.